

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

Отчёт о программном проекте на тему:

**«Построение эмбедингов пользователей по последовательности игровых событий на основе
контрастивного обучения, трансформеров и RNN»**

(часть проекта: метод RNN)

Выполнили:

студент группы БПМИ 245

Никонов Фёдор Алексеевич

31.05.2026

студент группы БПМИ 249

Гринюк Илья Олегович

31.05.2026

студент группы БПМИ 2414

Петраков Алексей Сергеевич

31.05.2026

Принял руководитель проекта:

Мамаев Александр Александрович

руководитель службы ML

ООО «Яндекс.Такси»

Москва, 2026

Содержание

Аннотация	4
Ключевые слова	4
1 Введение	5
2 Обзор литературы	5
3 Постановка задачи	6
3.1 Неформальная постановка	6
3.2 Формальная постановка	6
4 Данные и предобработка	7
4.1 Источник данных	7
4.2 Очистка и построение последовательностей	8
4.3 Разбиение выборки	8
4.4 Два режима построения эмбедингов	9
5 Метрики и downstream-оценка	9
6 Модель построения эмбедингов	10
6.1 Слой эмбедингов событий	10
6.2 Рекуррентный энкодер	10
6.3 Обучение через next-event prediction	10
6.4 Pooling скрытых состояний	10
7 Экспериментальное исследование	11
7.1 Next-event prediction	11
7.2 Prefix-based downstream retention	11
7.3 Pooling и capacity ablation	12
7.4 Supervised fine-tuning и negative controls	13
7.5 Full-history benchmark	14
7.6 Командное сравнение full-history подходов	15
7.7 Анализ пространства эмбедингов	16
8 Воспроизводимость и проверка корректности	17
9 Заключение	18

Аннотация

Работа посвящена построению эмбедингов пользователей мобильной игры по последовательностям игровых событий. Эмбединг должен сжимать историю действий пользователя в вектор фиксированной размерности, который можно использовать в downstream-задачах: прогнозировании удержания, анализе поведения и сравнении пользовательских профилей.

В этой части группового проекта исследуется подход на основе рекуррентных нейронных сетей. Были реализованы GRU- и LSTM-энкодеры, обучаемые без retention-разметки на задаче предсказания следующего события. После обучения скрытые состояния RNN используются для построения пользовательских эмбедингов. Полезность полученных представлений проверяется на задаче предсказания retention.

В работе рассмотрены два режима оценки. В prefix-based режиме модель получает только начальную часть истории пользователя, а retention-метка считается относительно конца префикса. Этот режим нужен для строгой проверки без утечки будущего поведения в признаки. В full-history режиме эмбединг строится по последним 512 событиям полной истории пользователя; он добавлен для сопоставления с общим форматом командных экспериментов.

RNN-модели значительно превосходят простые baseline для next-event prediction: на тестовой выборке GRU достигает MRR 0.8885 и Hit@10 0.9926. В строгой prefix-based downstream-проверке финальный кандидат **GRU h256, 1 layer, max pooling** даёт ROC-AUC 0.6922 ± 0.0052 для retention 7d и 0.7051 ± 0.0032 для retention 14d, улучшая табличный baseline. В согласованном full-history режиме RNN-эмбединги отдельно достигают ROC-AUC 0.9413 для retention 7d.

Ключевые слова

Эмбединги пользователей, последовательности событий, рекуррентные нейронные сети, GRU, LSTM, self-supervised learning, next-event prediction, retention prediction, prefix-based evaluation, downstream evaluation.

1 Введение

Поведение игрока в мобильной игре можно описать последовательностью событий: запуск игры, переходы между экранами, получение наград, взаимодействие с рекламой, игровые действия и другие элементы пользовательской активности. Такая последовательность содержит информацию о стиле игры, вовлеченности и вероятности возвращения пользователя.

Прямое использование сырых последовательностей в прикладных моделях неудобно. У разных пользователей сильно различается длина истории: в подготовленных данных медианная длина равна 83 событиям, а максимальная длина достигает 185 204 событий. Кроме того, порядок событий несёт смысл, который теряется при простом подсчёте частот. Поэтому полезно построить отображение, переводящее историю пользователя в компактный вектор фиксированной размерности.

Цель моей части проекта - построить такие пользовательские эмбединги с помощью RNN-моделей и проверить, сохраняют ли они полезный поведенческий сигнал. В отличие от supervised-модели retention, RNN-энкодер обучается на последовательной структуре самих событий: по префиксу истории он предсказывает следующий event. Retention используется позже только как внешний критерий качества эмбедингов.

Работа является частью группового проекта, в котором сравниваются три подхода к построению эмбедингов пользователей: SimCLR, BERT4Rec/Transformer и RNN. Настоящий отчет описывает RNN-часть: подготовку последовательностей, обучение GRU/LSTM, экспорт эмбедингов, downstream-оценку и дополнительные проверки устойчивости.

Основные результаты:

- реализован полный pipeline для RNN-эмбедингов: preprocessing, user-level split, next-event обучение, экспорт эмбедингов, downstream retention evaluation;
- GRU и LSTM сильно превосходят MostPopular и Markov-1 на next-event prediction;
- prefix-based протокол показывает, что RNN-эмбединги дают умеренный, но устойчивый прирост к табличным признакам в retention-задаче;
- проведены pooling, capacity, supervised fine-tuning, negative controls и seed-stability проверки;
- дополнительно построена full-history RNN-ветка на общем командном split для будущего сопоставления с SimCLR и BERT4Rec.

2 Обзор литературы

Рекуррентные нейронные сети являются классическим инструментом для моделирования последовательностей. LSTM была предложена Hochreiter и Schmidhuber [6] для борьбы с проблемой исчезающих градиентов в длинных последовательностях. Позже Cho и соавторы предложили GRU [4], более компактную рекуррентную архитектуру с gating-механизмом. Обе архитектуры применимы

к пользовательским событиям: модель последовательно обновляет скрытое состояние и тем самым сжимает историю пользователя в вектор.

В рекомендательных системах последовательное моделирование часто применяется для предсказания следующего действия или следующего объекта. Работа Hidasi и соавторов [5] показала, что GRU-подход может эффективно моделировать session-based поведение. Для более поздних transformer-based методов важной точкой сравнения является BERT4Rec [8], где последовательность элементов обучается через masked language modeling. В нашем групповом проекте BERT4Rec используется в другой ветке, а RNN-ветка служит более простой и интерпретируемой sequence-моделью.

Общая тема проекта связана с self-supervised представлениями. В этой области важны contrastive predictive coding и InfoNCE-подход [7], а также SimCLR [2], применяющий контрастивное обучение для построения представлений без ручной разметки. Эти работы относятся прежде всего к другим веткам проекта, но задают общий контекст: эмбединги обучаются на внутренней структуре данных, а затем проверяются на внешних задачах.

Качество эмбедингов удобно проверять через downstream-задачи, то есть внешние задачи, не использовавшиеся напрямую при self-supervised обучении. В этой работе основной downstream-критерий - предсказание retention. Для нелинейной оценки признаков используется XGBoost [3], так как градиентный бустинг хорошо работает с табличными признаками и позволяет сравнивать разные семейства эмбедингов в единой процедуре оценки. Общая идея representation learning как построения признаков, полезных для последующих задач, соответствует обзору Bengio и соавторов [1].

3 Постановка задачи

3.1 Неформальная постановка

Для каждого пользователя известна последовательность игровых событий, отсортированная по времени. Нужно построить модель, которая переводит эту последовательность в вектор фиксированной размерности. Пользователи с похожим поведением должны получать похожие векторы, а полученные эмбединги должны быть полезны в downstream-задачах.

Важная особенность работы: retention-разметка не используется как основной supervision для обучения энкодера. Энкодер обучается на задаче предсказания следующего события, а retention нужен для внешней проверки качества представлений.

3.2 Формальная постановка

Пусть пользователь u задан последовательностью событий

$$s_u = (x_1, x_2, \dots, x_{L_u}), \quad x_i \in V,$$

где V - словарь событий, а L_u - длина истории пользователя. Требуется обучить энкодер

$$f_\theta : V^* \rightarrow \mathbb{R}^d,$$

который переводит последовательность произвольной длины в эмбединг $z_u = f_\theta(s_u)$.

В RNN-подходе модель обучается на next-event prediction. Для обучающего окна $(x_1, \dots, x_t)$ модель предсказывает следующий токен x_{t+1} :

$$\hat{p}_{t+1} = \text{softmax}(Wh_t + b),$$

где h_t - скрытое состояние RNN после обработки первых t событий. После обучения скрытые состояния модели используются как представление пользователя.

Для downstream-проверки строится бинарная метка retention_h : был ли пользователь активен после горизонта h дней. В prefix-based режиме эта метка считается относительно конца префикса, а не относительно конца всей истории.

4 Данные и предобработка

4.1 Источник данных

Данные представляют собой логи AppMetrica мобильной игры. Каждая запись содержит идентификатор устройства пользователя, имя события, JSON-параметры события, timestamp и session id. Предобработка выполнялась на общем наборе логов проекта.

После очистки и фильтрации в RNN-ветке подготовлено 52 495 пользователей. Размер словаря событий после фильтрации составляет 81 токен. Распределение длин последовательностей имеет тяжелый хвост:

Таблица 1 – Характеристики подготовленных последовательностей RNN-ветки

Метрика	Значение
пользователей	52,495
размер словаря	81
минимальная длина	10
медианная длина	83

Метрика	Значение
средняя длина	972.12
95-й перцентиль	4,694
максимальная длина	185,204

4.2 Очистка и построение последовательностей

На этапе очистки удаляются записи с пустыми ключевыми полями, технические события и служебные debug/test/internal события. Далее события каждого пользователя сортируются по времени и row_id, чтобы получить стабильный порядок при совпадающих timestamp. Пользователи менее чем с 10 событиями исключаются.

Каждому имени события сопоставляется integer token, то есть целочисленный идентификатор события. Последовательность пользователя хранится как список таких токенов. Padding token с индексом 0 используется только для выравнивания длин последовательностей при подготовке batch для RNN.

4.3 Разбиение выборки

Финальные prefix-based эксперименты используют общий user-level split команды, импортированный из master_split_lesha.csv. Из 68 394 пользователей master split в RNN-подготовку попали 52 495 пользователей; все они присутствуют в master split.

Таблица 2 – Статистика последовательностей по частям user-level split

Split	Пользователей	Средняя длина	Медианная длина	Максимальная
				длина
train	36,760	956.73	84	131,726
val	7,896	984.92	79	63,480
test	7,839	1,031.39	86	185,204

Split проводится по пользователям, а не по событиям. Это исключает ситуацию, когда события одного пользователя одновременно попадают в train и test.

Для командного full-history сравнения отдельно использован согласованный протокол на SimCLR processed sequences. В нем RNN получает последние 512 событий для всех пользователей общего split: 47 875 train, 10 259 val и 10 260 test. Этот прогон не заменяет prefix-based эксперименты, а нужен только для сопоставления с SimCLR/BERT4Rec в retrospective full-history постановке.

4.4 Два режима построения эмбедингов

В работе используются два режима.

Первый режим - **prefix-based**. Модель строит эмбединг только по первым `prefix_len` событиям пользователя. Retention label считается относительно конца этого префикса. Такой режим отвечает на вопрос: можно ли по ранней истории пользователя построить представление, полезное для прогноза будущего поведения. Это строгая постановка без утечки будущих событий в признаки.

Второй режим - **full-history**. Модель строит эмбединг по последним 512 событиям полной доступной истории пользователя. Такой режим отвечает на другой вопрос: насколько хорошо RNN кодирует уже наблюдавшийся поведенческий профиль. Он добавлен для командного сопоставления с ветками, которые также строят эмбединги по полной истории.

Эти два режима не следует смешивать как один и тот же тип прогноза. Prefix-based результаты являются основной проверкой no-leakage predictive quality, а full-history результаты - отдельным retrospective benchmark.

5 Метрики и downstream-оценка

Для задачи next-event prediction используются MRR и Hit@K. MRR отражает средний обратный ранг правильного следующего события, а Hit@K - долю примеров, в которых правильное событие попало в первые K предсказаний.

Для downstream retention используются ROC-AUC и PR-AUC. ROC-AUC оценивает качество ранжирования положительных и отрицательных примеров, а PR-AUC дополнительно чувствителен к дисбалансу классов. Это важно для больших горизонтов retention, где доля положительного класса мала.

Downstream-оценка проводится через бинарный классификатор XGBoost. Модель обучается на train split, а метрики считаются на val/test. В stability-таблицах обучение повторяется на нескольких random seeds, после чего приводятся среднее значение и стандартное отклонение.

Baseline-признаки представляют собой простые агрегаты поведения пользователя. В prefix-based режиме они строятся только по тому же префиксу, что и RNN-эмбединг: длина префикса, число уникальных событий, доля повторов, число сессий, число активных дней и доли top-K частых событий. Top-K события выбираются только по train split. В full-history режиме аналогичные признаки строятся по последнему окну истории пользователя.

В таблицах ниже Baseline обозначает только эти агрегированные признаки, RNN embedding - только вектор RNN-энкодера, а Baseline + RNN embedding - их конкатенацию. В командном сравнении SimCLR/BERT/RNN используются результаты без дополнительных baseline-признаков, так как они

лучше отражают качество самих методов построения эмбеддингов.

6 Модель построения эмбеддингов

6.1 Слой эмбеддингов событий

На вход RNN подается последовательность целочисленных идентификаторов событий. Слой `nn.Embedding` переводит каждый token в плотный вектор размерности `event_dim`. В основных экспериментах используется `event_dim = 64`, padding token имеет индекс 0 и не несет информации о событии.

6.2 Рекуррентный энкодер

В работе реализован общий `RecurrentEncoder`, который поддерживает GRU и LSTM. Энкодер получает последовательность эмбеддингов событий и обновляет скрытое состояние по времени. Для обучения используется `pack_padded_sequence`, поэтому padding не влияет на скрытые состояния реальных событий.

Базовые модели:

Таблица 3 – Основные конфигурации RNN-энкодеров

Модель	event dim	hidden dim	layers	dropout
GRU baseline	64	128	1	0.0
LSTM baseline	64	128	1	0.0
Final GRU candidate	64	256	1	0.0

6.3 Обучение через next-event prediction

Модель обучается предсказывать следующий event. Для каждого обучающего окна входом является префикс, а целевой меткой - следующий token. На выходе RNN-проекция дает распределение по словарю событий.

Такая целевая функция не использует retention-разметку и поэтому подходит для построения универсальных поведенческих эмбеддингов. Если модель хорошо предсказывает следующий event, значит скрытое состояние содержит информацию о локальной структуре пользовательской истории.

6.4 Pooling скрытых состояний

Для получения одного вектора на пользователя из последовательности скрытых состояний используются несколько стратегий агрегации:

- last: последнее скрытое состояние;

- mean: среднее по всем реальным событиям;
- max: покоординатный максимум по реальным токенам, без учета padding;
- last_mean: конкатенация last и mean.

Pooling ablation показала, что для downstream retention наиболее полезным оказался **max pooling**. Финальный кандидат использует **GRU h256, 1 layer, max pooling**.

7 Экспериментальное исследование

7.1 Next-event prediction

Сначала проверялось, умеют ли RNN-модели моделировать последовательность событий. Для сравнения использовались два простых baseline. MostPopular всегда ранжирует события по их частоте в train split и не учитывает историю конкретного пользователя. Markov-1 использует только последнее событие и ранжирует следующие события по эмпирическим вероятностям переходов первого порядка.

Таблица 4 – Качество моделей на задаче предсказания следующего события

Модель	Split	MRR	Hit@5	Hit@10
MostPopular	test	0.4510	0.7154	0.8781
Markov-1	test	0.8158	0.9408	0.9775
GRU	test	0.8885	0.9733	0.9926
LSTM	test	0.8886	0.9732	0.9927

GRU и LSTM дают почти одинаковое качество и заметно превосходят простые baselines. Это подтверждает, что рекуррентная модель извлекает из истории пользователя больше информации, чем частотная модель или переход первого порядка.

7.2 Prefix-based downstream retention

Основная downstream-проверка проводилась в prefix-based режиме. Для каждого пользователя берутся первые prefix_len событий. Признаки и RNN-эмбеддинги строятся только по этому префиксу. Пользователь включается в оценку по конкретному horizon только если у него есть полный префикс и горизонт полностью наблюдаем в данных.

Длина префикса выбиралась отдельной ablation-проверкой на common-valid подвыборке, где для всех значений prefix_len использовался один и тот же набор пользователей. Проверялись префиксы 25, 50, 75, 100, 150, 200 и 300 событий. На common-valid проверке prefix_len = 50 давал лучший результат для retention 7d, а prefix_len = 150 был сильнейшим вариантом для single-embedding retention

14d. Поэтому `prefix_len = 150` выбран как компромисс: он не является лучшим по всем single-run метрикам, но дает сильное качество на более длинном горизонте и сохраняет достаточно контекста для финальной stability-оценки.

Финальная stability-оценка проводилась на `prefix_len = 150` и пяти seed XGBoost: 11, 42, 77, 123, 202.

Таблица 5 – Финальная prefix-based stability-оценка RNN-эмбедингов

Target	Feature set	ROC-AUC	PR-AUC	Test users
retention_7d	Baseline	0.6609 ± 0.0022	0.4551 ± 0.0042	2,805
retention_7d	Baseline + GRU h256 l1 max	0.6922 ± 0.0052	0.4978 ± 0.0023	2,805
retention_14d	Baseline	0.6788 ± 0.0027	0.3412 ± 0.0043	2,303
retention_14d	Baseline + GRU h256 l1 max	0.7051 ± 0.0032	0.3644 ± 0.0037	2,303

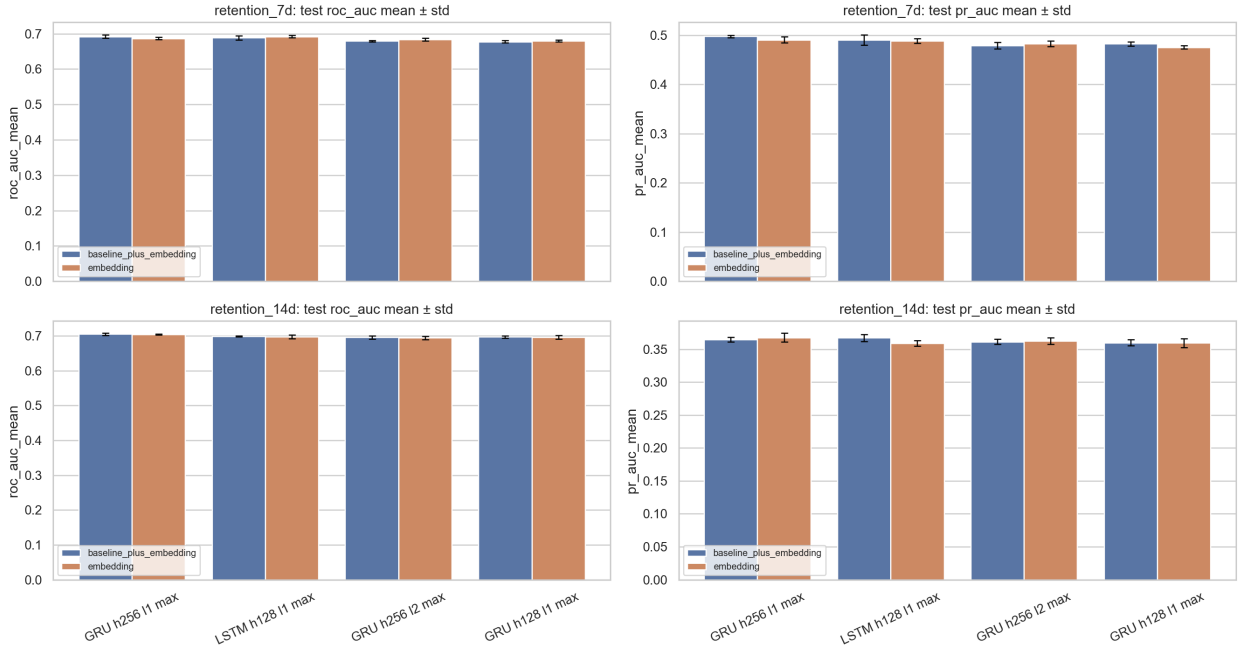


Рисунок 1 – Seed-stability финальных RNN-кандидатов на prefix-based retention.

Прирост к baseline составляет примерно +0.031 ROC-AUC для retention 7d и +0.026 ROC-AUC для retention 14d. Это умеренный, но устойчивый сигнал: RNN-эмбединг не заменяет простые агрегаты поведения, но добавляет к ним полезную последовательную информацию.

7.3 Pooling и capacity ablation

Pooling ablation проверяла, как способ агрегации скрытых состояний влияет на качество downstream-оценки. Для **GRU h128** лучшим вариантом без дополнительных baseline-признаков

оказался **max pooling**:

Таблица 6 – Лучшие результаты pooling ablation для GRU h128

Target	Лучший pooling setup	ROC-AUC	PR-AUC
retention 7d	GRU h128 max	0.6827	0.4800
retention 14d	GRU h128 max	0.7025	0.3663

Capacity sweep показал, что качество next-event prediction и качество downstream retention не полностью совпадают. **GRU h256 l2** дает лучший next-event MRR 0.8925, но финальным downstream-кандидатом стал **GRU h256 l1 max**, который лучше и стабильнее работает в retention-задаче.

Таблица 7 – Основные выводы capacity sweep

Experiment	Лучший вариант	Метрика
next-event capacity	GRU h256 l2	MRR 0.8925
retention_7d baseline + embedding	LSTM h128 l1 max	ROC-AUC 0.6909
retention_14d baseline + embedding	GRU h256 l1 max	ROC-AUC 0.7100
final stability	GRU h256 l1 max	выбран как основной кандидат

Финальный выбор делался не только по одному лучшему single-run числу, а по устойчивости и качеству на двух retention horizons.

7.4 Supervised fine-tuning и negative controls

Дополнительно проверялось, улучшает ли supervised fine-tuning GRU под retention качество относительно self-supervised next-event embeddings. Для retention 7d fine-tuned GRU достиг ROC-AUC 0.6882, что близко к лучшим self-supervised RNN-представлениям, но не меняет общий вывод: next-event обучение уже извлекает значительную часть retention-сигнала.

Negative controls нужны, чтобы проверить, что результат не объясняется случайной размерностью признаков или устройством XGBoost. В контрольных экспериментах:

- random embeddings дают ROC-AUC около 0.51 для retention 7d и около 0.50 для retention 14d;
- bag-of-events конкурентоспособен, но слабее основных RNN-представлений;
- shuffled-order embeddings сохраняют часть качества, что объясняется повторяемостью пользовательских префиксов, но уступают нормальным RNN-эмбедингам.

Таблица 8 – Negative controls для downstream retention

Feature set	Target	ROC-AUC	PR-AUC
Random embeddings	retention_7d	0.5104	0.2818
GRU mean	retention_7d	0.6756	0.4812
Baseline + GRU mean	retention_7d	0.6751	0.4757
Random embeddings	retention_14d	0.4982	0.1818
GRU mean	retention_14d	0.6937	0.3604
Baseline + GRU mean	retention_14d	0.6979	0.3663

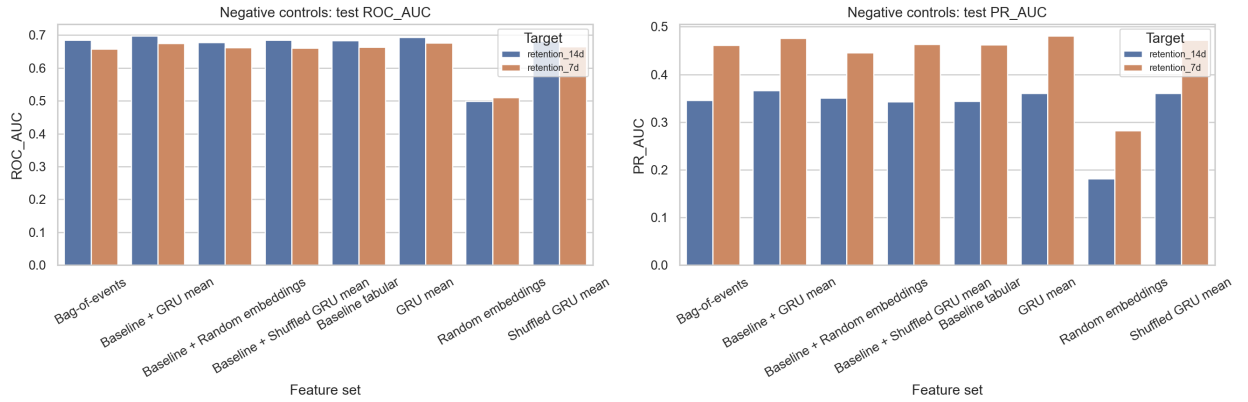


Рисунок 2 – Сравнение RNN-эмбеддингов с negative controls на тестовой части.

Эти проверки поддерживают вывод, что RNN-эмбеддинги несут реальный поведенческий сигнал.

7.5 Full-history benchmark

После prefix-based экспериментов была добавлена full-history RNN-ветка. Она использует тот же командный split, но строит эмбеддинг по последним 512 событиям полной истории пользователя. Для командного сравнения эта ветка была приведена к общему протоколу: RNN обучается на тех же SimCLR processed sequences, использует те же train/val/test user ids и те же duration labels из SimCLR-пакета.

Этот режим нужен для командного сопоставления с SimCLR/BERT4Rec, где другие ветки также работали с full-history представлениями. Его нельзя напрямую сравнивать с prefix-based результатами как с той же задачей раннего прогноза.

Таблица 9 – Full-history downstream benchmark RNN-эмбеддингов

Target	Feature set	ROC-AUC	PR-AUC	Test users
retention_1d	RNN embedding	0.9598 ± 0.0003	0.8818 ± 0.0012	10,260
retention_1d	Baseline + RNN embedding	0.9641 ± 0.0003	0.9005 ± 0.0010	10,260
retention_7d	RNN embedding	0.9413 ± 0.0005	0.6777 ± 0.0027	10,260
retention_7d	Baseline + RNN embedding	0.9471 ± 0.0006	0.7170 ± 0.0032	10,260
retention_14d	RNN embedding	0.9319 ± 0.0012	0.4960 ± 0.0089	10,260
retention_14d	Baseline + RNN embedding	0.9388 ± 0.0009	0.5565 ± 0.0076	10,260
retention_30d	RNN embedding	0.9658 ± 0.0027	0.1910 ± 0.0456	10,260
retention_30d	Baseline + RNN embedding	0.9757 ± 0.0014	0.2381 ± 0.0465	10,260

Метрики full-history значительно выше, потому что модель видит всю доступную историю пользователя. Это показывает, что RNN хорошо кодирует полный поведенческий профиль, но не заменяет prefix-based проверку, если интересует ранний прогноз будущего поведения. В согласованном прогоне используется та же тестовая часть на 10 260 пользователей, что и в командных full-history экспериментах.

7.6 Командное сравнение full-history подходов

В групповой части проекта, помимо RNN-ветки, были реализованы еще два подхода к построению пользовательских эмбеддингов. SimCLR-ветка обучала контрастивный encoder: две аугментированные версии истории одного пользователя сближались в пространстве эмбеддингов, а истории разных пользователей отталкивались друг от друга. Transformer/BERT4Rec-ветка обучала sequence encoder в masked/next-event постановке и затем экспортировала эмбеддинги по последним 512 событиям полной истории.

Для сопоставления подходов используется сравнение эмбеддингов без дополнительных baseline-признаков в общей full-history постановке. Метрики SimCLR взяты из экспортированных артефактов соответствующей ветки, метрики BERT - из итоговой сводки BERT-ветки, а метрики RNN - из согласованного full-history прогона. Во всех строках таблицы приведена оценка на 10 260 test-пользователях с одинаковыми долями положительного класса. Комбинированные варианты с ручными baseline-признаками не включаются в командную таблицу, поскольку они сильнее зависят от инженерии табличных признаков и хуже отражают качество самих эмбеддингов.

Таблица 10 – Командное full-history сравнение эмбеддингов без дополнительных baseline-признаков

Горизонт	Модель	ROC-AUC	PR-AUC	Пользователей	Доля positive
7d	BERT mean	0.9296	0.6228	10,260	11.1%
7d	RNN/GRU	0.9413	0.6777	10,260	11.1%
7d	SimCLR	0.9249	0.6263	10,260	11.1%
14d	BERT mean	0.9256	0.4467	10,260	6.0%
14d	RNN/GRU	0.9319	0.4960	10,260	6.0%
14d	SimCLR	0.9189	0.4974	10,260	6.0%
30d	BERT mean	0.9665	0.1500	10,260	0.34%
30d	RNN/GRU	0.9658	0.1910	10,260	0.34%
30d	SimCLR	0.8621	0.1725	10,260	0.34%

В текущей таблице, выровненной по full-history постановке, test split и retention-меткам, RNN/GRU дает лучший ROC-AUC среди результатов без дополнительных baseline-признаков на горизонтах 7 и 14 дней. На 30 днях BERT mean и RNN/GRU практически равны по ROC-AUC, а PR-AUC выше у RNN. При этом 30d нужно читать осторожнее: положительный класс составляет всего около 0.34% тестовой части.

Совпадение тестовой части, определения retention-меток и долей положительного класса делает таблицу содержательным сравнением для курсового проекта. При этом ветки реализовывались независимо, поэтому таблица не утверждает полного равенства downstream-кода во всех подходах.

7.7 Анализ пространства эмбеддингов

Для анализа пространства эмбеддингов были построены нормы, PCA-проекции и KMeans-профили. GRU и LSTM формируют невырожденные пространства представлений:

Таблица 11 – Диагностика пространства GRU и LSTM эмбеддингов

Модель	Mean norm	Median norm	PCA10 cumulative variance
GRU	8.4440	8.4336	0.6053
LSTM	6.9537	7.0392	0.5779

Первые 10 PCA-компонент объясняют около 60% дисперсии для GRU и около 58% для LSTM. Это говорит о выраженной структуре пространства. Кластеризация по GRU-эмбеддингам выделяет группы пользователей с различающимися profile statistics и retention rates.

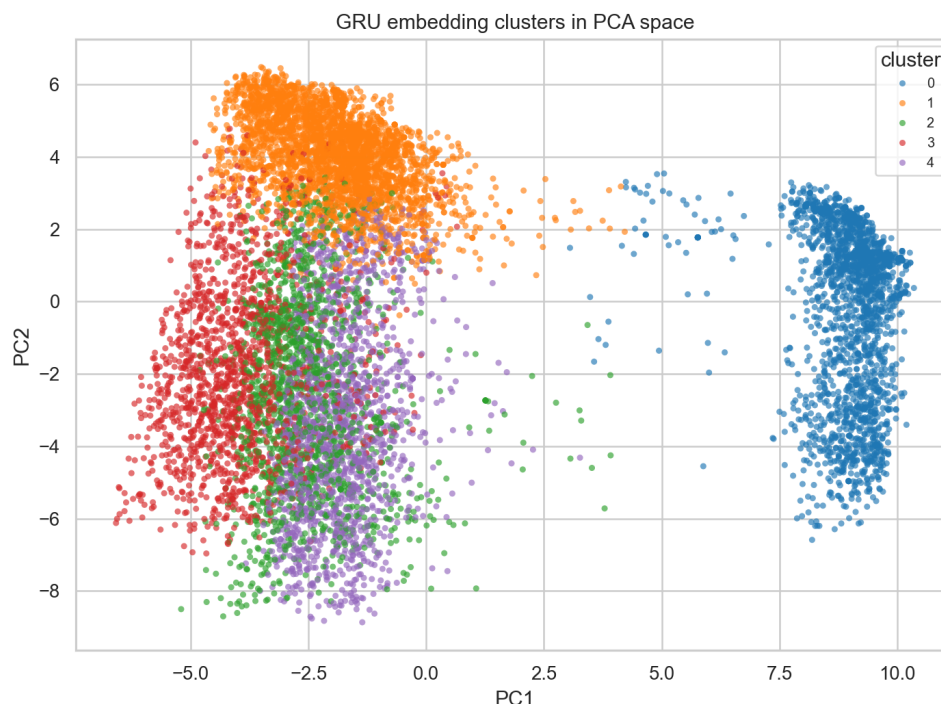


Рисунок 3 – PCA-проекция GRU-эмбеддингов с KMeans-кластерами пользователей.

8 Воспроизводимость и проверка корректности

Для снижения риска утечек и смешивания постановок в работе отдельно реализованы prefix-based и full-history сценарии. В prefix-based режиме все признаки, включая baseline-агрегаты и RNN-эмбеддинги, строятся только по доступному префиксу пользователя. В full-history режиме используется отдельная разметка и отдельный экспорт эмбеддингов по последнему окну истории.

Выбор top-K событий для baseline-признаков выполняется только на train split. Downstream-модели обучаются на train и оцениваются на val/test, что сохраняет user-level разделение данных. Для финальных таблиц используются несколько random seeds XGBoost, поэтому в отчете приводятся не только средние значения метрик, но и стандартные отклонения.

Экспериментальные артефакты проверяются отдельным gunner: он валидирует наличие ключевых таблиц, графиков, manifest-файлов и aligned full-history результатов, а также формирует сводную таблицу результатов. В текущей версии gunner проверяет 57 артефактов по 10 сериям экспериментов. Ноутбуки выполнены без сохраненных error outputs, а исходный код проходит синтаксическую проверку.

Таким образом, основная часть контроля качества относится не к структуре репозитория, а к корректности экспериментального протокола: отдельные режимы оценки, user-level split, отсутствие использования test split при построении baseline-признаков и воспроизводимая проверка итоговых

артефактов.

9 Заключение

В работе построены пользовательские эмбединги по последовательностям игровых событий с помощью RNN-моделей. GRU и LSTM обучались без retention-разметки на задаче next-event prediction, после чего скрытые состояния использовались как user embeddings.

Sequence-эксперименты показали, что RNN-модели значительно превосходят MostPopular и Markov-1 baselines. Это подтверждает, что модель извлекает из истории пользователя информацию о последовательной структуре событий.

Downstream-проверка в prefix-based режиме показала, что RNN-эмбединги дают устойчивый прирост к табличным признакам в задаче предсказания retention. Финальный кандидат **GRU h256, 1 layer, max pooling** достигает 0.6922 ± 0.0052 ROC-AUC для retention 7d и 0.7051 ± 0.0032 для retention 14d. Negative controls подтверждают, что этот результат не объясняется случайной размерностью признаков.

Full-history режим был добавлен отдельно для командного сопоставления. В согласованном прогоне RNN-эмбединги по полной истории достигают 0.9413 ROC-AUC для retention 7d и 0.9319 для retention 14d на 10 260 test-пользователях. Эта постановка интерпретируется как оценка качества представления полного поведенческого профиля, а не как строгий ранний прогноз будущего retention.

В общем full-history сравнении RNN/GRU показал лучший ROC-AUC без дополнительных baseline-признаков на горизонтах 7 и 14 дней среди текущих артефактов. SimCLR остается сильной контрастивной веткой, а BERT при согласованном определении retention-меток также показывает сильные метрики. Это подтверждает, что все три подхода извлекают поведенческий сигнал; при интерпретации таблицы важно учитывать, что ветки разрабатывались независимо, хотя приведены к общей постановке сравнения.

Итоговый вывод: RNN является простой и эффективной основой для построения пользовательских эмбедингов по игровым событиям. Она хорошо моделирует последовательность действий, дает полезный downstream-сигнал и может служить сильной базовой веткой для сравнения с SimCLR и BERT4Rec в общем проекте.

Список использованных источников

- [1] Bengio Y., Courville A., Vincent P. Representation Learning: A Review and New Perspectives // IEEE Transactions on Pattern Analysis and Machine Intelligence. 2013. Vol. 35, No. 8. P. 1798-1828. DOI: 10.1109/TPAMI.2013.50. URL: <https://doi.org/10.1109/TPAMI.2013.50>.
- [2] Chen T., Kornblith S., Norouzi M., Hinton G. A Simple Framework for Contrastive Learning of Visual Representations // Proceedings of the 37th International Conference on Machine Learning. PMLR. 2020. Vol. 119. P. 1597-1607. URL: <https://proceedings.mlr.press/v119/chen20j.html>.
- [3] Chen T., Guestrin C. XGBoost: A Scalable Tree Boosting System // Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. 2016. P. 785-794. DOI: 10.1145/2939672.2939785. URL: <https://doi.org/10.1145/2939672.2939785>.
- [4] Cho K., van Merriënboer B., Gulcehre C., Bahdanau D., Bougares F., Schwenk H., Bengio Y. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation // Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing. 2014. P. 1724-1734. DOI: 10.3115/v1/D14-1179. URL: <https://aclanthology.org/D14-1179/>.
- [5] Hidasi B., Karatzoglou A., Baltrunas L., Tikk D. Session-based Recommendations with Recurrent Neural Networks // ICLR. 2016. arXiv:1511.06939. URL: <https://arxiv.org/abs/1511.06939>.
- [6] Hochreiter S., Schmidhuber J. Long Short-Term Memory // Neural Computation. 1997. Vol. 9, No. 8. P. 1735-1780. DOI: 10.1162/neco.1997.9.8.1735. URL: <https://doi.org/10.1162/neco.1997.9.8.1735>.
- [7] van den Oord A., Li Y., Vinyals O. Representation Learning with Contrastive Predictive Coding. 2018. arXiv:1807.03748. URL: <https://arxiv.org/abs/1807.03748>.
- [8] Sun F., Liu J., Wu J., Pei C., Lin X., Ou W., Jiang P. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer // Proceedings of the 28th ACM International Conference on Information and Knowledge Management. 2019. P. 1441-1450. DOI: 10.1145/3357384.3357895. URL: <https://doi.org/10.1145/3357384.3357895>.